Part I — The plan: GitLab round with Karan Bhatia (Fri 2026-10-09)

Interview: Fri 2026-10-09. **Written:** Sat 2026-10-03 → 6 days left (Oct 3 – Oct 8, plus a warm-up on the 9th). **This replaces juggling three plans.** It merges (1) `GITLAB_WEEK_PLAN.md`, (2) the GitLab deep dive (`GITLAB_KARAN_BHATIA_DEEP_DIVE.md`, "DD"), and (3) the database/distributed-systems reading you did from Hello Interview and `MICROSOFT_DATABASES_TRANSACTIONS_DEEP_DIVE.md` ("DB doc") — now treated as **fundamentals he might probe**, not a separate interview.

> **Interviewer confirmed (2026-10-03): Karan Bhatia** — VP Eng & Engineering Fellow; ex-Azure AD identity, ex-New Relic multi-tenancy/observability. (An earlier "Kaul" was a slip.)

---

1. Why the database reading is still worth it for *this* round

He is not a coding-round interviewer and not a "gotcha" interviewer. But his background makes data-layer fundamentals **plausible follow-ups** whenever you describe a system:

His background	What it makes likely	Where you're covered
Azure AD / IAM, 5.5 yrs	tokens, composite identity, **consistency of permission changes** (revocation propagating)	DD §7.1; DB doc §3 (staleness), §8.4
New Relic — multi-tenant, NRDB (exabyte analytical engine)	**sharding, hot tenants, noisy neighbors, fairness**, partitioning, caching	DB doc §11; DD §5.3, §6.4
"Observability as the control system for agents"	telemetry pipelines, delivery semantics, idempotency	DD §4.8, §6.1; DB doc §8
Closed-loop agents (issue → fix → PR → validate)	**durable workflows, sagas, retries, exactly-once effects**	DD §1.10, §4.4, §6.8; DB doc §8

So: **DD first, DB doc as a "when he pokes at state/concurrency/scale, I can go one level deeper" layer.** You do **not** need the Microsoft-style SQL drills (window functions, query plans) for this round.

1b. Where everything is — this ONE PDF is the whole study set

This file is **Part I** of `GITLAB_MASTER_PREP.pdf`. Everything you need is in the same PDF; use the PDF bookmarks (left panel) to jump. "DD §n" = Part II.

Where	What it is	Read it for	Priority
Part I	This plan	The schedule and tiers	—
Part II (ch. 0–16)	GitLab deep dive for Karan Bhatia, incl. **ch. 15 Salesforce flows** and **ch. 16 narration playbook**	Everything Tier 1–2	★★★
Appendix G	Master study guide — everything Karan can ask from your resume (LangGraph, MCP, RAG, vector DBs, evals…)	Taught-from-scratch depth on each resume term	★★★
Appendix J2	Round-2 questions for your work Claude: telemetry/metrics + data modeling, contention, sagas, orchestration vs choreography, scaling, sharding, caching, per project	Closing the ownership gaps (Tier 1 #3)	★★★ (do today)

Where	What it is	Read it for	Priority
Appendix I	GitLab interview question bank	Out-loud drilling, Oct 8 mock	★★★
Appendix J	Questions to take to your work-environment Claude	Closing the ownership gaps (Tier 1 #3)	★★★ (do today)
Appendix H	Concepts study guide (permission gate, etc.)	If a concept doesn't click	★★
Appendix A	Aniruddha deep dive — Agentforce for Setup internals	Salesforce architecture, security, scale	★★
Appendix K	Database & transactions fundamentals	Tier 3 follow-ups only	★
Appendix O	Original detailed week plan (per-day reading lists, self-checks)	Detail behind §3's day table	★★
Appendix B	ResolveFlow design docs (checkpointing, CAP, deployment, feedback, secrets)	Your own system's data/consistency/deploy answers	★★
Appendix C / D / E	Harness patterns · enterprise guide · RAG basics	Reference, as needed	★
Appendix F	Agentforce Handbook (public platform, MCP, grounding)	Reference only — do not read cover to cover	★

Removed 2026-10-03: the LangGraph Deep Dive, the evals/observability/DR guide and the two Agent-Runtime guides (former Appendices L–N) are no longer in this PDF — their Downloads originals were deleted before the PDF was rebuilt. Equivalent coverage: Appendix G §1.1 (LangGraph), §1.2 (MCP), Part II §4.7–4.8 (evals, observability).

Not in this PDF on purpose: `High-Level_Design_Agentforce_for_Setup_v2.pdf` (a Salesforce internal design document — keep it where it is, don't redistribute it), your resumes, and `Backend Preparation Doc.pdf` (your own notes).

---

2. Priority tiers — what to do with limited time

Tier 1 — MUST (non-negotiable; each is a way to lose this round)

#	Item	Why	Source
1	**Resolve the benchmark claim.** "50-scenario, multi-provider, 86% routing accuracy, 24% lower cost" is **not** in the public repo (only 11 safety-gate tasks + 6 capability checks, OpenAI only, rule-based classifier). Decide: find the real one / soften the resume line / prepare the honest-scope answer.	A Fellow can check it in 60 s; credibility risk	DD §0.3
2	**Infra/deployment axis, with OWN / PATTERN / THIN stated.** AWS (Lambda vs ECS), CI/CD end to end, Kubernetes, observability/on-call, Data Cloud pipeline.	The exact gap that cost you CrowdStrike	DD §14, `feedback_deployment_infra_gap`
3	**Fill the Salesforce ownership table** (OWN / PARTIAL / CONSUMED + one citation each). Only you can do this — it needs your work environment.	Knowing *which parts you touched* was the CrowdStrike gap, not content	DD §15.15, `OFFICE_CLAUDE_QUESTIONS.md`, `~/Desktop/salesforce_architecture_probe.md`

#	Item	Why	Source
4	**The three bridges in his vocabulary**: permission-aware execution ↔ Azure AD; evals ↔ observability for agent correctness; multi-tenant access mgmt ↔ his specialty. Say `traceId` / `tenantId` / composite identity naturally.	He'll hear your fit in the first 10 minutes	DD §0.2, §15.7, §16.6
5	**Narrate the flows cleanly** (the eight-beat template) for Agentforce for Setup, the Slack agent, ResolveFlow — 60 s and 3 min versions, out loud.	"Describe the flow well" was your ask	DD §15–16

Tier 2 — SHOULD (strong signal; do after Tier 1)

#	Item	Source
6	**Whiteboard designs:** the closed loop (his reference architecture); 500 agents / one monorepo; multi-tenant agent execution platform; "why did the agent open this MR?" audit trace	DD §1.10, §8.1–8.7
7	**STAR stories C and D** (confident-wrong-answer incident; the paused approval that vanished) + A/B	DD §3
8	**Defend every resume number** with the five-question template	DD §2.1–2.2
9	**GitLab's own vocabulary:** composite identity, remote flows, AI Catalog, merge trains, Gitaly, Sidekiq, Duo Agent Platform	DD §1.2, §7, §11.6
10	**Security for code-pushing agents:** trust zones, prompt injection, composite identity, CI controls	DD §4.2, §4.11, §7
11	**Evals for code agents:** pass@k vs pass^k, statistics, online evals	DD §4.7

Tier 3 — DATA-LAYER FUNDAMENTALS (your Hello Interview reading — "if he goes there")

Learn each to the level of **"state the invariant, the race, the fix, the limit"** — enough to answer one follow-up well, not to lecture.

Topic	Status (your report)	One-line target	DB doc
Transactions / ACID / isolation anomalies	read in doc; *not drilled*	Define lost update, write skew; know snapshot ≠ serializable	§1–3
Dealing with contention	✅ done	Atomic conditional write → locks → OCC → serializable; deadlock prevention by lock ordering	§5–6
Multi-step processes (sagas)	✅ done	Orchestration vs choreography; idempotent compensations; why 2PC blocks	§8
Idempotency / outbox / exactly-once *effect*	in doc	at-least-once + idempotent = exactly-once effect	§8.3–8.4
Sharding	✅ done *(shaky — re-check)*	shard key criteria; what breaks across shards; logical-shard resharding	§11.2
Caching	✅ done	patterns, stampede, invalidation, never the source of truth	§11.3
Data modeling / indexing	✅ done	composite order, covering, SARGable	§7, §10
Consistent hashing	⬜ left	ring + vnodes; ~1/N keys move on change	—
CAP / PACELC	⬜ left	C = linearizability; consistency vs latency even w/o partition	§11.1
Scaling reads / writes	⬜ left	replicas+lag, caches, batching, queues, hot-key splitting	§11
Numbers to know	⬜ left	RAM ~100 ns · SSD ~100 µs · DC RTT ~0.5 ms · cross-region ~50–150 ms	§11.4
PostgreSQL deep dive	❓ unconfirmed	what A/C/I/D each rely on	§1–4

Topic	Status (your report)	One-line target	DB doc
Temporal, CDC	⬜ left	Temporal ≠ LangGraph (event-history replay vs snapshot); CDC = WAL → events	§8
NoSQL / Kafka / Redis / ZooKeeper deep dives	⬜ left	**drop for this round** unless time remains	—

For this round, finish only: CAP, consistent hashing, scaling reads/writes, numbers (≈3 h total), plus confirm PostgreSQL's ACID appendix. **Skip** the NoSQL/Kafka/Redis/ZooKeeper deep dives, the SQL drills, and the Microsoft problem breakdowns — none move this interview.

Tier 4 — only if time remains

Ruby/Rails literacy (DD §11 — conversational only); mentoring/leadership phrasing (DD §10); reading list.

---

3. Day-by-day (merged with `GITLAB_WEEK_PLAN.md`)

Daily shape: 90-minute blocks; end each day by answering the self-check **out loud, no notes** and logging anything shaky in `GAPS.md`.

Day	Main block (Tier 1–2)	Data-layer block (Tier 3, ≤ 90 min)	End-of-day check
Sat Oct 3 (today)	**Honesty audit + ownership.** Decide the benchmark answer (Tier 1 #1). Start filling the ownership table; send the work-Claude probe questions. Re-read DD §0 and §2.1.	CAP + consistent hashing (the two short, near-certain ones).	Say your benchmark answer in one breath. Define CAP's C vs ACID's C.
Sun Oct 4	**Your systems at implementation depth:** ResolveFlow code walk (DD §2.3–2.4), then Salesforce flows (DD §15.2–15.7). Narrate each with the eight beats.	Scaling reads + writes; numbers to know.	3-minute Setup-agent narration; 3-minute ResolveFlow narration, recorded.
Mon Oct 5	**Infra axis:** DD §14 end to end — AWS, CI/CD, K8s, on-call trace. Mark every claim OWN/PATTERN/THIN honestly.	PostgreSQL ACID appendix; skim DB doc §3 (isolation) + §5 (contention).	For each infra topic: "my role is ***; the mechanism is*** ."
Tue Oct 6	**Agent framework depth:** orchestration, durability (LangGraph vs Temporal — DB doc §8 + the comparison table), guardrails, evals, observability (DD §4). Bridges in his vocabulary (DD §15.7, §16.6).	Sagas/outbox/idempotency (DB doc §8).	Explain why a checkpointer isn't Temporal; pass@k vs pass^k.
Wed Oct 7	**Whiteboards:** DD §8.1–8.7, timed 35 min each, aloud, drawing the trust-boundary line first. Scalability + distributed-systems mechanics (DD §5–6).	Sharding re-check (the thing you missed): what breaks across shards; logical-shard resharding; multi-tenant noisy neighbors (DD §5.3).	Two full whiteboards without notes.
Thu Oct 8	**Mock day:** full 45–60 min mock using DD §13's likely flow; STAR stories C/D; questions to ask him (DD §12). Drain `GAPS.md`.	One pass of "isolation anomalies + deadlock prevention" aloud.	Finish the mock; list the 3 weakest answers and fix them.
Fri Oct 9	**Warm-up only.** Re-read §0.2 bridges, the closing litmus test, your questions for him. **No new material.**	none	Sleep.

If you fall behind, **cut Tier 3 first, then Tier 4. Never cut Tier 1.**

---

4. The one-page "if he goes data-layer" cheat sheet

Say the *invariant → race → tool → limit* order every time:

- **Concurrency:** single-row invariant → atomic conditional `UPDATE`; multi-row → `SELECT … FOR UPDATE` (lock in a consistent order); low contention → optimistic version column; invariant spans rows you only *read* (**write skew**) → serializable or lock what you read. Isolation levels name which anomalies you can't see, not how it's built. Snapshot isolation still allows write skew.
- **Across services:** you can't wrap them in one transaction → **saga** with idempotent compensations, or **durable execution**; avoid 2PC (blocking, in-doubt). Dual-write → **transactional outbox** + CDC. Exactly-once effect = at-least-once delivery + idempotent consumer.
- **Scale:** exhaust indexes/caching/replicas before sharding; shard key = high cardinality + even + matches the dominant query; sharding breaks joins, cross-shard transactions, unique constraints, global indexes; reshard via many logical shards. Consistent hashing moves ~1/N keys on a change, vs ~all with `% N`.
- **Consistency:** CAP's C is linearizability; it bites only during a partition; PACELC adds latency vs consistency otherwise. Caches and replicas are *stale by design* — never the source of truth for a permission or an invariant (his IAM home turf: revocation propagation).
- **Multi-tenancy (his specialty):** `tenant_id` on everything and leading every index, row-level security, per-tenant quotas/fairness, noisy-neighbor isolation (bulkheads), pin large tenants to dedicated shards.

---

5. Open items only you can close

1. ~~Confirm the interviewer's name~~ — confirmed: Karan Bhatia.
2. The benchmark decision (Tier 1 #1).
3. The Salesforce ownership table + the eight Headless 360 capability names (work environment).
4. Whether the Microsoft round is *also* scheduled — if so, tell me its date; this plan assumes it isn't a priority this week.
5. Which Hello Interview items you've actually finished (I'll update the tracker in the DB doc).